lab1.md

Laboratoire 1 (GTI700 - volet applicatif)

IoT et services web

L'objectif de ce laboratoire est de concevoir un service web et une application web simples pour exposer les données provenant d'un capteur (dans le cas présent, nous utiliserons le capteur de température fourni avec le kit *Sunfounder*) rattaché à un périphérique Raspberry Pi.

Lors des laboratoires de la première partie de la session, vous avez utilisé davantage le langage Python. Dans cette seconde partie, nous allons utiliser principalement le langage JavaScript au moyen de l'environnement d'exécution `Node.JS`.

Dans ce laboratoire, vous utiliserez un ou deux Raspberry Pi (si vous n'avez qu'un seul, remplacez le second périphérique par un ordinateur - voir la section *Déploiement* pour plus de détails). Vous allez d'abord installer une version récente de `Node.JS`. Par la suite, vous allez devoir écrire un court module de code JavaScript permettant d'obtenir les données des capteurs de température et d'humidité à intervalle réguliers. Vous aurez ensuite à construire un serveur web rudimentaire en JavaScript (back-end), qui sera déployé sur le premier Raspberry Pi (A), et qui exposera un service simple permettant d'obtenir les données récoltées des capteurs selon une API donnée (back-end). Finalement, vous construirez une page HTML (front-end) qui sera fournie par le serveur web s'exécutant sur le second Raspberry Pi ou sur un ordinateur (B), et qui permettra d'afficher de manière visuelle les données à l'utilisateur, sur un autre ordinateur ou tablette (C).

Les laboratoires sont en équipe de 3. De plus, puisqu'il y a plusieurs volets à ce laboratoire, nous vous suggérons de répartir les différentes tâches selon les expertises des membres de l'équipe (p. ex., gestion des Raspberry Pis, back-end, front-end, etc.). Tous les membres de l'équipe doivent participer de manière équitable aux laboratoires.

0- Installation de Node.JS sur le Raspberry Pi

La première étape consiste à installer l'environnement Node.JS sur le(s) Raspberry Pis (et sur l'ordinateur B si nécessaire). Malheureusement, il est possible que les dépôts officiels `apt` du système d'exploitation (Ubuntu ou Raspbian) contiennent une version plus ancienne de `Node.JS`. Pour installer une version plus récente, nous vous recommandons d'utiliser un script fourni par `nodesource` qui ajoute un dépôt `apt` permettant l'installation facile de la dernière version de `Node.JS` : https://github.com/nodesource/distributions#debinstall

Testez ensuite le bon fonctionnement en tapant la commande suivante. La sortie devrait afficher `vW.XY.Z` -- vérifier que cela correspond à la version installée.

```
node -v
```

Finalement, pour installer des librairies Node.JS supplémentaires, nous recommandons d'utiliser le gestionnaire de paquets `npm` afin d'effectuer une installation locale plutôt qu'installer via les dépôts Ubuntu, puisque ces derniers risquent de contenir des versions obsolètes.

1- Obtention des données des capteurs via les pins GPIO

Pour brancher correctement et lire les données des capteurs de température et d'humidité, veuillez vous référer aux laboratoires (4 à 6) du volet 1 (réseau) du cours. Vous devez choisir les capteurs appropriés pour accomplir cette tâche. De plus, des exemples de code en Python (p. ex., `temperature.py`, `humid.py`) étaient fournis lors des laboratoires du premier volet du cours -- nous vous suggérons de partir de ces exemples comme base de référence.

Vous devez écrire un module simple en JavaScript qui permettra de maintenir un historique des valeurs de température mesurées (ex., avec un intervalle de 3 secondes), ainsi que la dernière valeur lue. Deux options s'offrent à vous pour récupérer les valeurs du capteur:

1. Vous pouvez réécrire le code source en Python en JavaScript en utilisant une librairie telle que `rpi-gpio` afin de pouvoir accéder aux pins GPIO du Raspberry Pi. Cette approche est plus complexe d'un point de vue implémentation.
2. Une approche possiblement plus simple consiste à lancer les programme Python (p. ex., `temperature.py`, `humid.py`) à partir de votre code JS (vous pouvez les modifier légèrement afin qu'ils lisent les valeurs à la fréquence voulue et qu'ils produisent en sortie les valeurs dans le format désiré), et en faisant en sorte que votre module JavaScript intercepte le flux de sortie afin de traiter les valeurs lues. Un exemple de code est fourni en annexe A.

Pour fins de correction, votre module JS devrait pouvoir être exécuté de façon indépendante et imprimer les valeurs de température lues au fur et à mesure dans la console (**tâche 1.1**). Nous validerons cela à la correction.

2- Implémentation d'un service web simple (*back-end*)

Vous allez implémenter ici un service web rudimentaire en JavaScript qui sera déployé sur le Raspberry Pi A. Votre serveur web devrait accepter les requêtes HTTP sur le port **8080**. Ce dernier va également intégrer le module d'obtention des données des capteurs (partie 1). Vous devez implémenter un total de 6 points d'entrée.

Pour la température:

- Si l'URL demandée par le client est `/capteurs/temp` (p. ex., `http://adresseIPduPI:8888/temp`), votre serveur devrait retourner la dernière valeur de température lue. Exemple: `21.4`
- Si un paramètre supplémentaire est encodé dans l'URL (p. ex., `/capteurs/temp/5`), le serveur devrait retourner un historique des `n` valeurs passées sous forme de tableau JSON. Chaque élément du tableau sera un tuple de type `[timestamp, value]` (`timestamp` réfère à la date/heure où la valeur a été lue). Exemple: `[ ["2024/10/27 21:00:01", 21.4], ["2024/10/27 21:00:04", 21.2], ["2024/10/27 21:00:07", 21.7], ["2024/10/27 21:00:11", 21.8], ["2024/10/27 21:00:14", 22.1] ]`. Note: le format exact du `timestamp` peut varier et est laissé à votre discrétion, tant que l'information est correcte. Si moins de `n` données sont disponibles, le tableau de retour devrait retourner toutes les valeurs disponibles et leurs timestamps. Vous pouvez assumer que le nombre maximal de données demandées sera 720 (une heure), donc vous devriez purger l'historique au besoin.
- Enfin, si un paramètre supplémentaire `moy` est ajouté à l'URL précédente (p. ex., `/capteurs/temp/5/moy`), vous devez retourner la **moyenne** des `n` températures (p. ex., 20.5). Si moins de `n` données sont disponibles, vous calculerez plutôt la moyenne sur l'ensemble des données disponibles.

- Vous devez suivre exactement le même principe pour les données d'humidité: les 3 points d'entrée seront les mêmes (p. ex., `/capteurs/hum` , `/capteurs/hum/5` , `/capteurs/hum/5/moy`). Vous allez plutôt retourner le pourcentage d'humidité avec une décimale (p. ex., `75.4%`).

Afin d'implémenter votre back-end (serveur web), nous recommendons l'utilisation du cadriciel `Express.JS` ; ceci étant dit, vous êtes toutefois libres d'utiliser tout autre cadriciel de votre choix qui vous permettra d'implémenter les requis demandés. Le cas échéant, il sera de votre responsabilité de savoir l'utiliser. Attention à la surconception -- les exigences du back-end ne sont pas d'une grande complexité!

À titre informatif, l'annexe B présente un exemple de code qui peut vous aider (utilisant le cadriciel `Express.JS`).

3- Implémentation d'une page web simple pour visualiser les données des capteurs (*front-end*)

La dernière partie du laboratoire consiste à intégrer une page HTML simple qui affichera les données météo (température et humidité). Vous allez implémenter un autre serveur web rudimentaire en JavaScript qui sera déployé sur le Raspberry Pi B (ou sur l'ordinateur B, si vous n'avez qu'un seul PI), et qui écoutera sur le port **8889**. La page fournie devra contenir deux colonnes - la première colonne affichera les données de température, tandis que la deuxième affichera les données d'humidité (vous pouvez chercher un *template* CSS pour effectuer la mise en page à 2 colonnes, mais vous devez mentionner la source).

Pour chacune des deux colonnes (température et humidité), vous devez afficher:

- la valeur actuelle
- la moyenne des 30 dernières valeurs
- un historique des 30 dernières valeurs lues sous la forme d'un tableau à deux colonnes: (1) le *timestamp* (date et heure) de la dernière mesure et (2) la valeur mesurée
- un graphique contenant des points reliés par des des lignes, qui permet d'afficher visuellement les 30 dernières valeurs (l'axe des X affiche le temps (heure:minute:seconde), tandis que l'axe des Y affiche la valeur, soit la température ou l'humidité). Plus d'informations sont données ci-bas concernant les graphiques.

Note: pour les moyennes, les tableaux et les graphiques, si moins de 30 valeurs sont disponibles dans l'historique, vous devez considérer uniquement l'ensemble des valeurs disponibles.

La page sera disponible à l'URL suivante: `/affichage` . Elle devra obtenir les données en consommant les APIs implémentées au back-end (section 2 du labo), **côté-client** (au moyen de requêtes de type AJAX ou *fetch* dans le code JS du front-end) -- cela vous permettra également de rafraîchir facilement l'ensemble des valeurs affichées à intervalles réguliers. **Vous devez rafraîchir toutes les données à chaque 3 secondes, sans rafraîchir la page dans son entièreté - vous devez plutôt utiliser du code JavaScript pour rafraîchir les valeurs et mettre à jour les graphiques**.

Notez qu'il est important de bien gérer les conditions d'erreurs pouvant survenir lors de la transmission de la requête: ceci est une bonne pratique en général, mais est d'une grande importance dans le cas présent où on héberge un serveur web sur le Raspberry Pi connecté en réseau WiFi. En cas d'erreur, vous devez afficher un message à cet effet sur la page, et vous devez continuer de tenter d'obtenir les nouvelles données selon l'intervalle demandé. Veuillez vous assurer de tester votre gestion des erreurs (p. ex., en arrêtant le processus `node` , ou encore en désactivant le WiFi sur le PI), puis en restaurant le service (votre chargé de laboratoire en fera autant à l'évaluation).

Conseil: la page `/affichage` devra servir du contenu HTML au client, ainsi que du code JavaScript pour obtenir les données du serveur et rafraîchir la page. Vous aurez également possiblement à servir du code CSS. Libre à vous d'inclure le code CSS et JS dans votre page HTML directement, ou de les "servir" à partir de fichiers JS ou CSS externes.

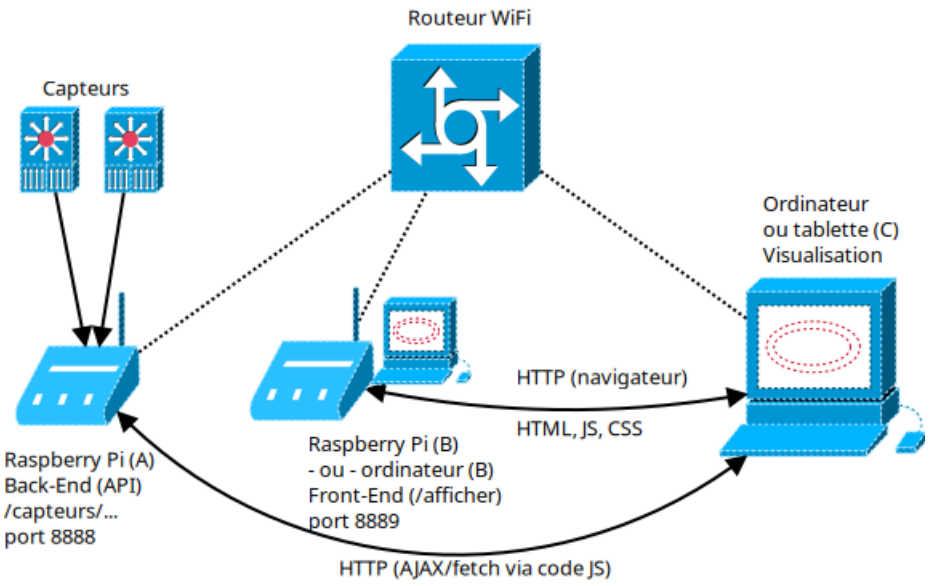
Pour vous aider, en annexe C, des exemples de code JavaScript sont fournis.

Précisions concernant les graphiques

Pour les deux graphiques, vous devez utiliser une librairie (nous recommendons ChartJS: https://www.chartjs.org/). L'exemple suivant peut vous servir de référence (vous pouvez regarder son code source): https://www.chartjs.org/samples/latest/charts/line/basic.html comme modèle de graphique. Chaque graphique contiendra une seule série (données de température ou de température). Vous devez afficher un historique "roulant" des 30 dernières valeurs obtenues du back-end. Vous devez ajuster les axes afin que les données soient lisibles et qu'on puisse constater une variation. L'affichage devrait être dynamique: lorsqu'une donnée arrive, vous l'ajoutez au "dataset" de votre graphique, et ce dernier est redessiné.

Déploiement

Le déploiement des différents composants devrait respecter le diagramme suivant. Vous devez déployer le "back-end" (APIs -- `/capteurs/...`) sur le Raspberry Pi A et le "front-end" (`/affichage`) sur le Raspberry Pi B (ou sur l'ordinateur B si vous n'avez qu'un seul PI).



Rapport

Pour ce TP, il n'y aura pas de rapport.

Grille de correction

Tâche	Description	Points max.
0.1	Installation de Node.JS	2

Tâche	Description	Points max.
1.1	Module JS pour obtenir les données de température et humidité	6
2.1	API: dernière valeur de température	2
2.2	API: historique des dernières valeurs de température (JSON)	3
2.3	API: moyenne des dernières valeurs de température	3
2.4	API: dernière valeur d'humidité	2
2.5	API: historique des dernières valeurs d'humidité (JSON)	3
2.6	API: moyenne des dernières valeurs d'humidité	3
3.1	Page: température actuelle	2
3.2	Page: moyenne des températures	2
3.3	Page: tableau historique des températures	5
3.4	Page: graphique des températures	5
3.5	Page: humidité actuelle	2
3.6	Page: moyenne des valeurs d'humidité	2
3.7	Page: tableau historique des valeurs d'humidité	5
3.8	Page: graphique des valeurs d'humidité	5
3.9	Page: formattage adéquat, en deux colonnes	3
Pénalité	Non respect du déploiement	-25%
Pénalité	Pas de rafraîchissement automatique des données	-30%
Pénalité	Gestion incorrecte des erreurs AJAX	-10%
Pénalité	Autres pénalités, au besoin	
Démo	Professionnalisme/préparation adéquate (modulation de la note globale)	<5>
Total		Max. 55

Note: il est essentiel que la démonstration au chargé soit préparée de manière professionnelle. Une note sur 5 sera attribuée à ce critère, et cette note modulera votre note finale: (5 multipliera votre note par 100%, 4 la multipliera par 90%, 3 la multipliera par 80%, 2 la multipliera par 70%, 1 la multipliera par 60% et 0 la multipliera par 50%).

Remise

La remise du code source se fera sur Moodle/ENA. Voir les instructions et la date de remise sur Moodle/ENA.

Code source

Bien que le code ne soit pas évalué d'office, nous nous réservons quand même le droit de le vérifier, et **le code complet devra être soumis avec la remise.** Toutefois, nous nous attendons à ce que vous mettiez des bonnes pratiques de conception et d'implémentation en oeuvre.

Utilisation d'un entrepôt Git

Pour le projet de session, vous devez utiliser un entrepôt Git (GitHub ou GitLab disponible à l'ÉTS) pour votre projet. Tous les membres de l'équipe devraient "pousser" du code pour étayer la contribution de tous et chacun. Advenant le cas où nous recevions une plainte concernant un membre de l'équipe ne fournissant pas sa juste part de travail, nous pourrions vous demander de nous donner accès à votre entrepôt ou de nous fournir une copie des "logs" (bien entendu, nous espérons fortement que cela ne se sera pas nécessaire!).

Politique sur le plagiat et l'utilisation d'intelligence artificielle

Tel qu'énoncé dans le plan de cours:

> Les clauses du « Règlement sur les infractions de nature académique de l'ÉTS » s'appliquent dans ce cours ainsi que dans tous les cours du département. Les étudiants doivent consulter le Règlement sur les infractions de nature académique (https://www.etsmtl.ca/A-propos/Direction/Politiques-reglements/Infractions_nature_academique.pdf) pour identifier les actes considérés comme étant des infractions de nature académique ainsi que prendre connaissance des sanctions prévues à cet effet. À l'ÉTS, le respect de la propriété intellectuelle est une valeur essentielle et les étudiants sont invités à consulter la page Citer, pas plagier ! (https://www.etsmtl.ca/Etudiants-actuels/Baccalaureat/Citer-pas-plagier).

Nous attirons votre attention sur le fait qu'il est **interdit de réutiliser, en tout ou en partie**, le travail d'une autre équipe ou réalisé à une session antérieure. Nous conservons l'ensemble des travaux des sessions antérieures et avons accès au code source soumis, et nous nous réservons le droit d'utiliser des outils de comparaison du code source et de détection de plagiat. Tout plagiat détecté sera sanctionné selon les politiques en vigueur à l'ÉTS.

De plus, tel qu'énoncé dans le module 0 (introduction du volet 2):

> Les outils de type "SIAG" représentent une évolution formidable (ex., ChatGPT, Copilot, Gemini, Meta AI). Leur utilisation doit toutefois être encadrée dans un contexte pédagogique, puisque l'objectif est de former les étudiant.e.s à devenir des ingénieur.e.s compétent.e.s et qualifié.e.s. Il est donc nécessaire de développer un bonne compréhension des concepts et technologies vues dans le cours.
>
> - Laboratoires: L'utilisation des SIAG pour poser des questions de programmation ponctuelles et très spécifiques est tolérée, mais chaque utilisation doit être déclarée de manière spécifique dans les rapports.

Annexes

Annexe A: exécution et capture de la sortie standard

Cet extrait de code montre comment exécuter un processus et capturer sa sortie standard au moyen du module `child_process.spawn` . L'extrait de code est tiré du fil `StackOverflow` suivant: https://stackoverflow.com/questions/10232192/exec-display-stdout-live

```
var spawn = require('child_process').spawn,
    ls    = spawn('ls', ['-lh', '/usr']);

ls.stdout.on('data', function (data) {
  console.log('stdout: ' + data.toString());
});

ls.stderr.on('data', function (data) {
  console.log('stderr: ' + data.toString());
});

ls.on('exit', function (code) {
  console.log('child process exited with code ' + code.toString());
});
```

Annexe B: serveur HTTP simple au moyen de `Express.JS`

Cet extrait de code montre comment initialiser un serveur HTTP simple au moyen du module `express` , traiter une requête simple sur une URL donnée, capturer des paramètres, et écrire une réponse dans le flux de sortie.

```
// app.js

const express = require('express')
const app = express()
const port = 3000

// Requêtes simples

app.get('/', (req, res) => res.send('Hello World!'))

app.get('/page.html', (req, res) => res.send({ajson: "object"})

// La fonction app.sendFile() permet d'envoyer un fichier

/*
L'exemple suivant démontre la possibilité de capturer des paramètres:

Route path: /users/:userName
Request URL: http://localhost:3000/users/JohnSmith
req.params: { "userName": "JohmSmith"}
*/

app.get('/users/:userName', function (req, res) {
  res.send(req.params)
})

app.listen(port, () => console.log(`Example app listening at http://localhost:${port}`))
```

Pour exécuter: `node app.js`

Annexe C: Requête AJAX

C.1 - Au moyen de l'API classique (`XMLHttpRequest`)

Cet exemple de code montre comment ouvrir une connexion HTTP et effectuer une requête asynchrone sur un serveur web en utilisant l'API AJAX standard du W3C. Cet extrait de code est tiré des exemples du cours GTI525 (ÉTS).

```
function processResponse(obj) {
  // ...
}

var url = "/ajax";

var xhr = new XMLHttpRequest();
xhr.open("GET", url);
xhr.onload = function() {
  // Fonction invoquée si une réponse est obtenue

  if (xhr.status==200) {
    // Code 200: réponse OK
    if (xhr.getResponseHeader("Content-Type") == "application/json") {
      // Au besoin, on peut vérifier l'en-tête Content-Type pour vérifier que la sortie correspond au type
attendu (par exemple, JSON)

      // Parser la réponse au format JSON pour obtenir un objet JS (result)
      var result = JSON.parse(xhr.responseText);
      processResponse(result);
    }
    else {
      console.log("Invalid content type");
    }
  } else {
    // Code de réponse différent de 200: indique possiblement une erreur (p. ex, 404 = page introuvable)
    console.log("Invalid response code");
  }
}
xhr.ontimeout = function() {
  // La requête n'a pas abouti après un certain délai
  console.log("Timed out");
}
xhr.onerror = function() {
  // Il y a eu une erreur réseau (par exemple, le serveur n'a pas répondu)
  console.log("Resulted in an error !");
};
xhr.onabort = function() {
  // Sera invoquée seulement si la requête est annulée par l'invocation de la méthode "abort" de l'objet
XMLHttpRequest
  console.log("Request aborted!");
};

// Transmet la requête (de manière asynchrone)
xhr.send();
```

C.2 - Au moyen de l'API `fetch`

Cet exemple de code montre comment ouvrir une connexion HTTP et effectuer une requête asynchrone sur un serveur web en utilisant l'API `fetch` du W3C. Cet extrait de code est tiré des exemples du cours GTI525 (ÉTS).

En utilisant la syntaxe des fonctions fléchées en JavaScript ES6+

```
// Fonction adaptée de: https://www.tjvantoll.com/2015/09/13/fetch-and-errors/
handleErrors = (response) => {
    if (!response.ok) throw Error(response.status + ": " + response.statusText);
    return response;
}

// Obtenir les titres des dernières publications sur le Reddit etsmtl
fetch("https://www.reddit.com/r/etsmtl.json")
        .then(handleErrors) // Attraper les erreurs de protocoles HTTP
        .then(response => response.json()) // Si le contenu est en format JSON, sinon si le contenu est en
format texte: return response.text()
        .then(data => { // Traiter les données, le format de data correspondant au format de réponse
demandé
    let children = data.data.children
    for (let child of children) {
      let title = child.data.title
      console.log(title)
    }
        }).catch(error => console.log("Une erreur est survenue: " + error) );
  // Attrape les erreurs réseau, et de protocole grâce à l'appel de handleErrors dans la chaîne de
promesses
```